

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Аппаратное и программное обеспечение сетей» за 5 семестр
Тема: «Организация UDP – сервера и UDP – клиента»

Выполнил:
Студент ФЭИС
Группы ИИ-28
Клименко М.С.
(1 подгруппа)

Проверил:
Кулик А.Д.

Брест 2026

Вариант 9

Цель: изучить теорию о сетевых протоколах, сокетах, организации сервера и клиента.

Задание:

Вариант задания для бригады студентов	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Номер задания для реализации TCP (UDP) сервера (см. табл. 2).	1	2	3	4	5	6	7	8	1	3	2	4	6	5	8	7
Номер задания для реализации TCP (UDP) клиента (см. табл. 3).	8	7	6	5	4	3	2	1	4	3	2	1	8	7	6	5

UDP – сервер:

1	Отсылка клиенту содержимого текстового файла <имя файла> в случае приема сервером в потоке символов команды load fname.txt, <имя файла> – имя некоторого текстового файла, находящегося в каталоге сервера. В случае, если запрашиваемый файл отсутствует в каталоге сервера, сервер должен отослать сообщение об этом и разорвать соединение.
---	--

UDP – клиент:

Вариант	Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих*: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: disconnect <адрес> <порт>	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: connect <адрес> <порт> 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
1	Home	1), 2)	+	2)
2	PgUp	1), 3)	-	1)
3	End	2), 3)	+	1)
4	PgDn	1), 2), 3)	-	2)
5	Home	1), 3)	+	1)
6	PgUp	2), 3)	-	1)
7	PgDn	1), 2)	+	2)
8	End	1), 2), 3)	-	1)

Код сервера:

```
import java.io.*;
import java.net.*;

public class Server {
    public static final int PORT = 9090;
```

```

public static void main(String args[]) {
    try (DatagramSocket socket = new DatagramSocket(PORT)) {
        System.out.println("UDP Server started: " + socket);

        byte[] buf = new byte[1024];

        while (true) {
            DatagramPacket packet = new DatagramPacket(buf, buf.length);
            socket.receive(packet);

            String received = new String(packet.getData(), 0,
packet.getLength()).trim();
            InetAddress clientAddress = packet.getAddress();
            int clientPort = packet.getPort();

            System.out.println("Received from " + clientAddress + ":" +
clientPort + " -> " + received);

            if (received.startsWith("load ")) {
                String targetFileName = received.substring(5).trim();
                File file = new File(targetFileName);

                if (file.exists() && file.isFile()) {
                    try (BufferedReader fileReader = new BufferedReader(new
FileReader(file))) {
                        String fileLine;
                        while ((fileLine = fileReader.readLine()) != null)
{
                            byte[] data = (fileLine + "\n").getBytes();
                            DatagramPacket sendPacket = new
DatagramPacket(data, data.length, clientAddress, clientPort);
                            socket.send(sendPacket);
                        }

                        byte[] eofData = "--- EOF ---\n".getBytes();
                        socket.send(new DatagramPacket(eofData,
eofData.length, clientAddress, clientPort));
                    }
                } else {
                    String errMsg = "Error: file '" + targetFileName + "'
not found on server!\n";
                    byte[] data = errMsg.getBytes();
                    socket.send(new DatagramPacket(data, data.length,
clientAddress, clientPort));
                    System.out.println("File not found. Client notified.");
                }
            } else if (received.equals("exit")) {
                System.out.println("Client requested exit.");
                String msg = "Connection closed.\n";
                byte[] data = msg.getBytes();
                socket.send(new DatagramPacket(data, data.length,
clientAddress, clientPort));
            }
        }
    } catch (IOException e) {
        System.err.println("UDP Server error: " + e.getMessage());
    }
}

```

Код клиента:

```
import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Client {
    public static final String HOST = "127.0.0.1";
    public static final int PORT = 9090;

    private static PrintWriter logWriter;
    private static DatagramSocket socket;
    private static InetAddress serverAddress;
    private static final DateTimeFormatter dtf =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public static void main(String[] args) {
        try {
            logWriter = new PrintWriter(new FileWriter("protocol.txt", true),
true);
        } catch (IOException e) {
            System.err.println("Log file creation error: " + e.getMessage());
        }

        JFrame frame = new JFrame("UDP Client (Send via PgDn)");
        JTextArea textArea = new JTextArea();
        textArea.setEditable(false);
        JTextField textField = new JTextField();

        frame.add(new JScrollPane(textArea), BorderLayout.CENTER);
        frame.add(textField, BorderLayout.SOUTH);
        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

        try {
            socket = new DatagramSocket();
            serverAddress = InetAddress.getByName(HOST);
            log("CONNECTION STARTED with " + HOST + ":" + PORT);
            textArea.append("Connected! Enter a command and press PgDn to
send.\n");

            new Thread(() -> {
                try {
                    byte[] buf = new byte[1024];
                    while (true) {
                        DatagramPacket packet = new DatagramPacket(buf,
buf.length);

                        socket.receive(packet);
                        String line = new String(packet.getData(), 0,
packet.getLength()).trim();
                        log("RECEIVED: " + line);
                        textArea.append("Server: " + line + "\n");

                        textArea.setCaretPosition(textArea.getDocument().getLength());
                    }
                }
            })
        } catch (Exception e) {
            log("Exception: " + e.getMessage());
        }
    }
}
```

```

        } catch (IOException e) {
            textArea.append("Connection closed.\n");
        }
    }).start();
} catch (IOException e) {
    textArea.append("Connection error: " + e.getMessage() + "\n");
    return;
}

Action sendAction = new AbstractAction() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String text = textField.getText();
        if (text.isEmpty()) return;

        try {
            byte[] data = (text + "\n").getBytes();
            DatagramPacket packet = new DatagramPacket(data,
data.length, serverAddress, PORT);
            socket.send(packet);

            log("SENT: " + text);
            textArea.append("You: " + text + "\n");
            textField.setText("");

            if (text.equals("exit")) {
                closeConnection();
                System.exit(0);
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
};

textField.getInputMap(JComponent.WHEN_FOCUSED)
    .put(KeyStroke.getKeyStroke(KeyEvent.VK_PAGE_DOWN, 0), "send");
textField.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW)
    .put(KeyStroke.getKeyStroke(KeyEvent.VK_PAGE_DOWN, 0), "send");
textField.getActionMap().put("send", sendAction);

frame.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        closeConnection();
    }
});

textField.requestFocusInWindow();
}

private static void log(String message) {
    String time = dtf.format(LocalDateTime.now());
    String logEntry = "[" + time + "] " + message;
    System.out.println(logEntry);
    if (logWriter != null) {
        logWriter.println(logEntry);
    }
}

```

```
private static void closeConnection() {  
    try {  
        log("CONNECTION ENDED");  
        if (socket != null && !socket.isClosed()) socket.close();  
        if (logWriter != null) logWriter.close();  
    } catch (Exception ex) {  
        ex.printStackTrace();  
    }  
}
```

Вывод: Изучил теорию о сокетах, сетевых протоколах и реализовал UDP – сервер и UDP – клиент.